

Container mit Docker, Service Discovery & Konfigurationsspeicher



Das Problem dynamischer IP-Adressen

Warum brauchen wir Service Discovery?

Das Problem in der Cloud:

In einer klassischen Umgebung hatten Server feste IP-Adressen (z. B. 192.168.1.50 für die Datenbank).

In Docker- und Microservice-Umgebungen sind IP-Adressen ephemeral (flüchtig). Wenn ein Container abstürzt, neu startet oder hochskaliert wird, bekommt er jedes Mal eine neue, zufällige IP-Adresse.

Die Frage: Wie soll das Backend wissen, unter welcher IP-Adresse es die Datenbank oder den Noten-Service aktuell erreicht? Man kann die IPs nicht mehr fest in den Code schreiben (Hardcoding)!

Container mit Docker, Service Discovery & Konfigurationsspeicher

Die Lösung – Was ist Service Discovery?

Das Prinzip von Service Discovery

Kernbotschaft: Ein automatisches, hochverfügbares Erkennungssystem, das wie ein Echtzeit-Telefonbuch funktioniert.

Der Ablauf in zwei Schritten:

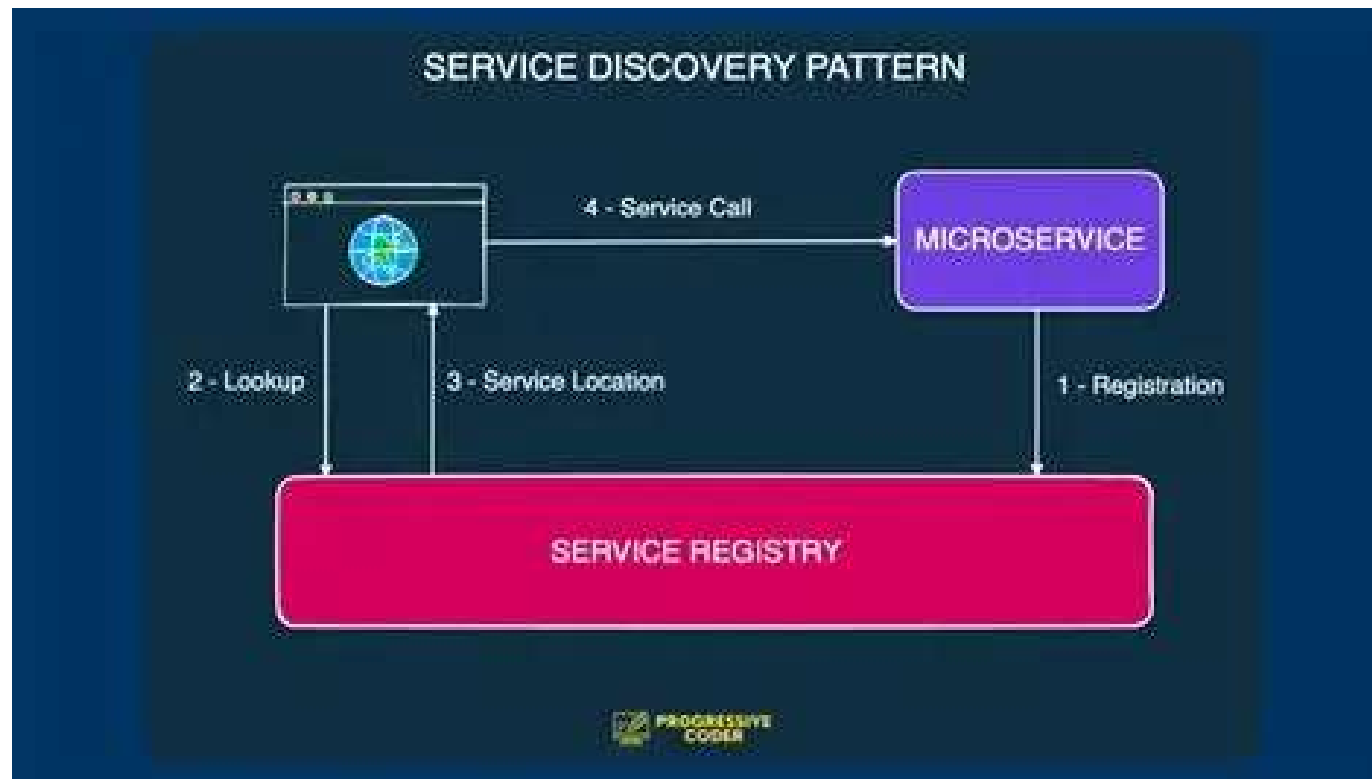
Service Registration (Anmeldung): Sobald ein Container startet, trägt er sich selbst (oder ein Hilfswerkzeug) mit seinem Namen und seiner aktuellen IP/Port in das Verzeichnis ein.

Service Discovery (Abfrage): Will ein anderer Container diesen Dienst aufrufen, fragt er das Verzeichnis nach dem Namen (z. B. "noten-service") und erhält die aktuell gültige IP..

Container mit Docker, Service Discovery & Konfigurationsspeicher

Die Lösung – Was ist Service Discovery?

Das Prinzip von Service Discovery Quelle: linkedIn, Saurabh Dashora



Container mit Docker, Service Discovery & Konfigurationsspeicher

Key-Value Stores (Verteilte Konfigurationsspeicher)

Wo werden diese Daten gespeichert?

Kernbotschaft: Service-Discovery-Tools nutzen im Hintergrund sogenannte Distributed Key-Value Stores (verteilte Schlüssel-Wert-Datenbanken).

Eigenschaften dieser Speicher:

Einfache Struktur: Daten werden nicht in komplexen Tabellen (wie bei SQL), sondern als einfache Pfade gespeichert (z. B. /services/database -> 10.0.1.5:5432).

Verteilt (Distributed): Sie laufen als Cluster auf mehreren Servern gleichzeitig. Fällt ein Server aus, sind die Daten immer noch da.

Konsistenz: Sie garantieren, dass alle Server im Cluster exakt denselben Zustand des Telefonbuchs kennen.

Container mit Docker, Service Discovery & Konfigurationsspeicher

Key-Value Stores (Verteilte Konfigurationsspeicher)

Wo werden diese Daten gespeichert?

Kernbotschaft: Service-Discovery-Tools nutzen im Hintergrund sogenannte Distributed Key-Value Stores (verteilte Schlüssel-Wert-Datenbanken).

Eigenschaften dieser Speicher:

Einfache Struktur: Daten werden nicht in komplexen Tabellen (wie bei SQL), sondern als einfache Pfade gespeichert (z. B. /services/database -> 10.0.1.5:5432).

Verteilt (Distributed): Sie laufen als Cluster auf mehreren Servern gleichzeitig. Fällt ein Server aus, sind die Daten immer noch da.

Konsistenz: Sie garantieren, dass alle Server im Cluster exakt denselben Zustand des Telefonbuchs kennen.

Container mit Docker, Service Discovery & Konfigurationsspeicher

Wichtige Zusatzfunktion – Health Checking

Health Checks: Das Telefonbuch aktuell halten

Stichpunkte:

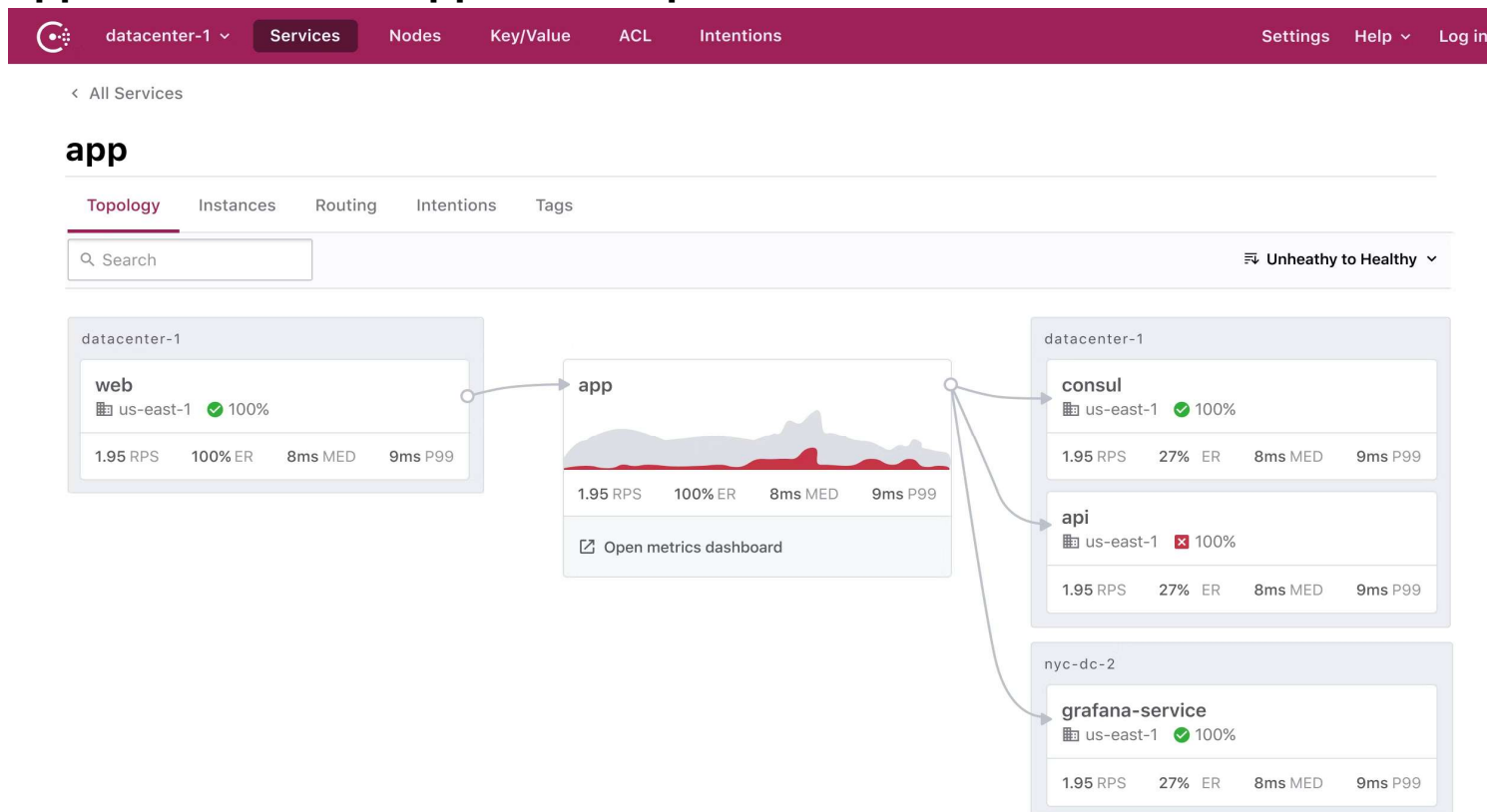
Was passiert, wenn ein Container unbemerkt abstürzt? Er stünde immer noch mit seiner alten IP im Telefonbuch.

Lösung: Das Service-Discovery-System führt kontinuierlich Health Checks (Gesundheitsprüfungen) durch. Es pingt den Container regelmäßig an oder fragt eine Test-URL ab.

Antwortet der Container nicht mehr, wird er sofort automatisch aus dem Telefonbuch gelöscht, damit kein anderer Microservice mehr versucht, die tote IP aufzurufen.

Container mit Docker, Service Discovery & Konfigurationsspeicher

Wichtige Zusatzfunktion – Health Checking Health Checks: Das Telefonbuch aktuell halten
Quelle: <https://app-fox.com/en-US/app/hashicorp-consul/>



Container mit Docker, Service Discovery & Konfigurationsspeicher

Bekannte Werkzeuge im Vergleich Die Tools des Docker-Ökosystems

Übersicht der bekanntesten Implementierungen:

etcd: Extrem schlank, in Go geschrieben. Fokus liegt auf einem absolut sicheren Key-Value-Speicher. (Wird im Hintergrund von Kubernetes genutzt).

Consul: Ein All-in-One-Tool von HashiCorp. Bringt Service Discovery, ein fortgeschrittenes Web-Interface für Admins und integriertes Health Checking direkt mit.

ZooKeeper: Der altgediente Klassiker aus dem Apache-Hadoop-Umfeld. Sehr robust, benötigt aber eine Java-Laufzeitumgebung und gilt im modernen Docker-Umfeld oft als etwas zu schwerfällig.

Container mit Docker, Networking & Kommunikation

Die Standard-Infrastruktur (Single-Host Networking)

Wie kommunizieren Container auf einem Server?

Kernbotschaft: Wenn Docker auf einem einzelnen System (wie unserem Rocky Linux) installiert wird, baut es automatisch eine virtuelle Netzwerkinfrastruktur auf.

Die Standard-Komponenten:

Die **Docker-Bridge** (docker0): Ein virtueller Switch im Server. Jeder neu gestartete Container wird standardmäßig an diese Bridge angeschlossen.

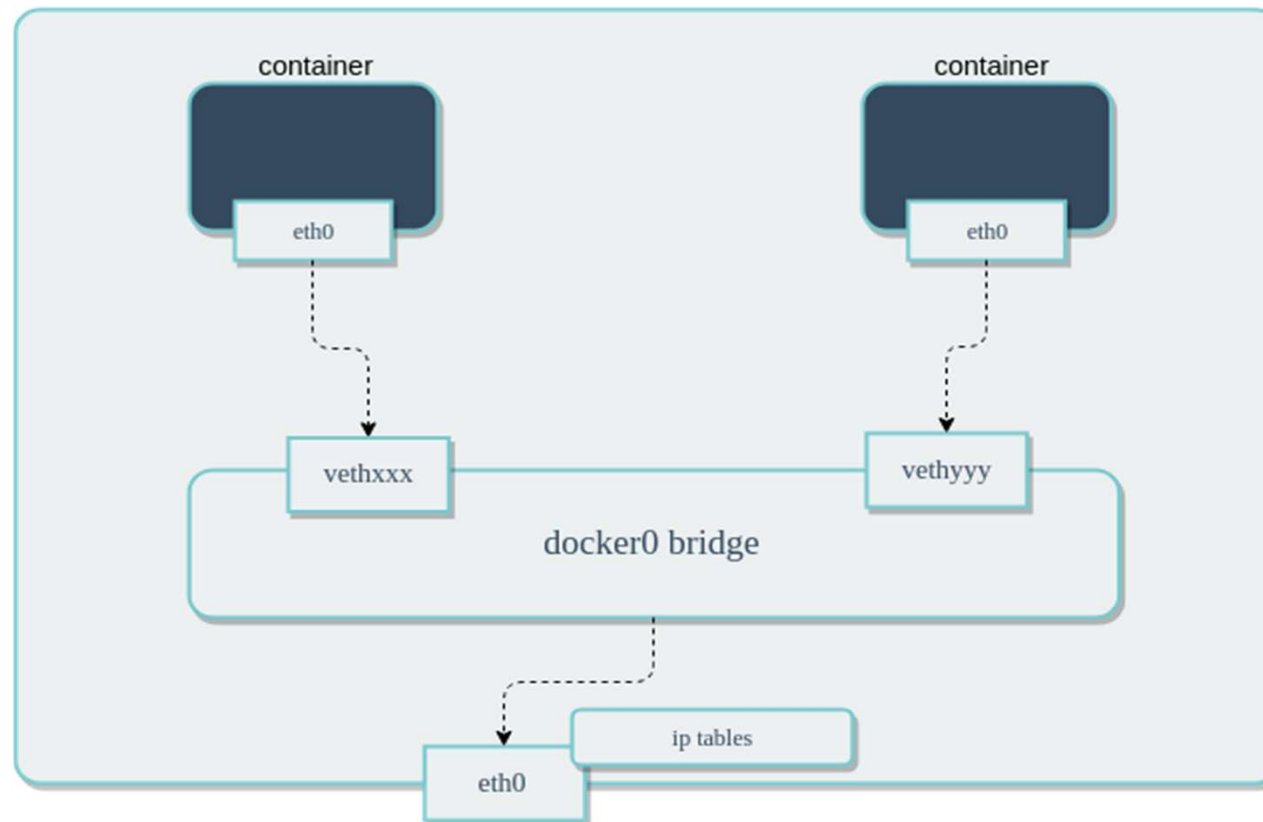
Virtuelle Interfaces (veth): Wie ein unsichtbares Netzkabel. Ein Ende steckt im Container, das andere Ende in der docker0-Bridge.

Zuweisung: Jeder Container erhält eine private IP-Adresse (meist aus dem Bereich 172.17.x.x). Sie können sich untereinander direkt über diese IPs anpingen.

Container mit Docker, Networking & Kommunikation

Die Standard-Infrastruktur (Single-Host Networking)

Wie kommunizieren Container auf einem Server? Quelle: https://www.suse.com/c/rancher_blog/introduction-to-container-networking/



Container mit Docker, Networking & Kommunikation

Der Weg nach draußen (Port-Mapping & NAT)

Wie wird ein Container aus dem Internet erreichbar?

Problem: Die internen IPs (172.17.x.x) sind von außen (außerhalb des Rocky Linux Servers) komplett unsichtbar und nicht erreichbar.

Die Lösung: **Port-Mapping (NAT):**

Docker nutzt die Linux-Firewall (iptables), um Ports des echten Servers an den Container weiterzuleiten.

Beispiel: Ein Angular-Frontend läuft im Container auf Port 80. Wir mappen es auf den Server-Port 8080.

Rufst du nun `http://dein-rocky-linux:8080` auf, leitet der Server die Anfrage blitzschnell an den Container auf Port 80 weiter.

Container mit Docker, Networking & Kommunikation

Die Herausforderung – **Multi-Host Networking**

Das Problem mit mehreren Servern

Das Szenario: Unsere Microservices wachsen. Der User-Service läuft auf Server A, der Noten-Service auf Server B.

Das Problem: * Beide Server haben ihre eigene docker0-Bridge.

Beide vergeben unabhängig voneinander IPs (z. B. haben beide einen Container mit der IP 172.17.0.2).

Die Container können sich nicht direkt erreichen, weil die Bridges nichts voneinander wissen und die IPs kollidieren.

Container mit Docker, Networking & Kommunikation

Die Lösung – Overlay-Netzwerke

Overlay-Netzwerke: Virtuelle Tunnel im Netz

Kernbotschaft: Ein Overlay-Netzwerk baut ein riesiges, virtuelles, flaches Netzwerk über alle physischen Server hinweg.

Funktionsweise:

Die echten Servergrenzen werden für die Container komplett unsichtbar.

Wenn Container A (Server 1) ein Paket an Container B (Server 2) schickt, wird dieses Paket vom Networking-Tool in ein ganz normales Datenpaket verpackt (Kapselung / Encapsulation).

Dieses Paket wandert durch das normale Schul- oder Firmennetzwerk zum anderen Server. Dort wird es entpackt und dem Ziel-Container übergeben.

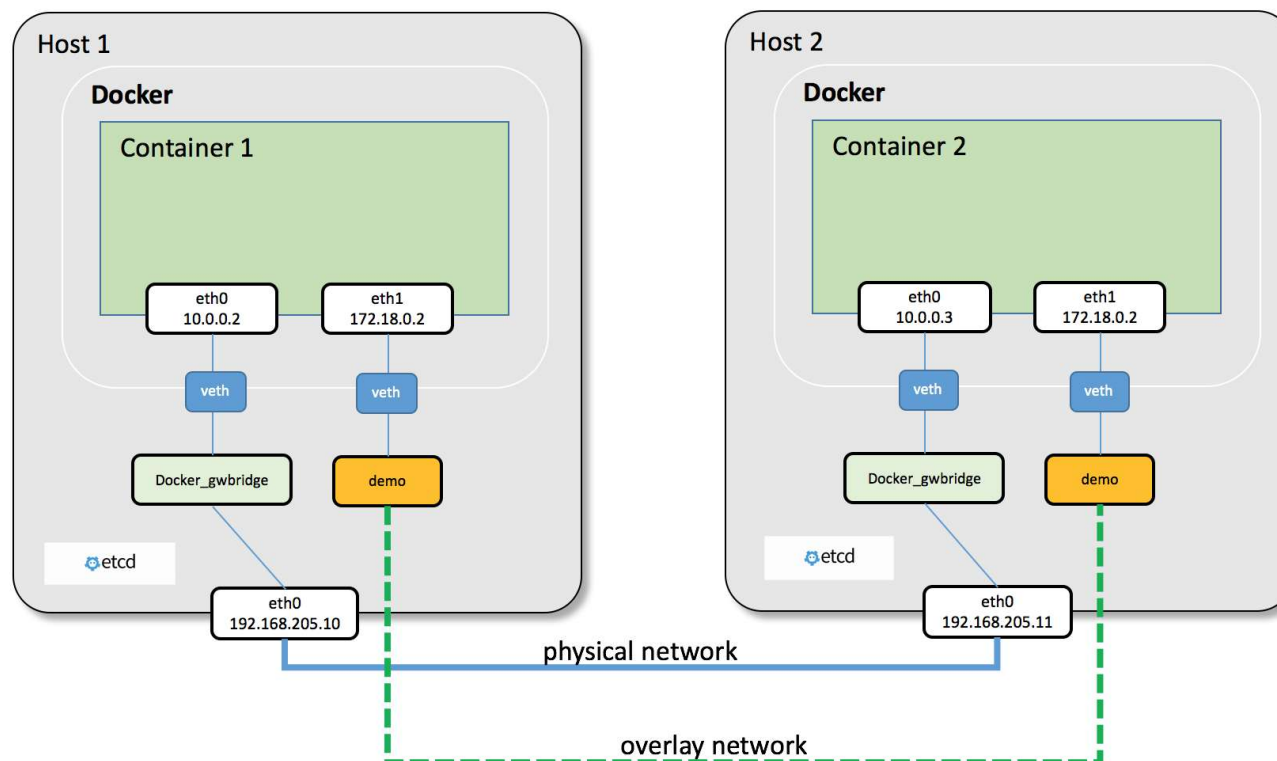
Bekannte Protokolle: VXLAN (wird sehr häufig genutzt).

Container mit Docker, Networking & Kommunikation

Die Lösung – Overlay-Netzwerke

Overlay-Netzwerke: Virtuelle Tunnel im Netz

Quelle: <https://mohittalniya.medium.com/overview-of-docker-networking-lets-connect-the-containers-39b34249ad91>



Container mit Docker, Networking & Kommunikation

Die Architekten des Netzwerks (Tools)
Bekannte Networking-Tools im Docker-Umfeld

Die wichtigsten Implementierungen:

docker-overlay (Integrierte Lösung): Das hauseigene Tool von Docker, wenn man den Docker Swarm Mode nutzt. Extrem einfach einzurichten.

Flannel: Ein sehr schlankes, von CoreOS entwickeltes Overlay-Netzwerk, das oft in Kubernetes -Einsteiger-Clustern genutzt wird (nutzt meist UDP oder VXLAN).

Calico: Das Hochleistungs-Tool für Profis. Es kann nicht nur Tunnel bauen, sondern bringt mächtige Network Policies (Sicherheitsregeln) mit. Damit kannst du verbieten, dass das Frontend direkt mit der Datenbank spricht, selbst wenn sie im selben Netzwerk liegen.

Container mit Docker, Scheduling & Orchestrierung

Die Herausforderung im großen Maßstab (Scale)

Warum brauchen wir Orchestrierung?

Das Szenario: Eine Anwendung besteht aus 50 verschiedenen Microservices. Einige davon müssen zur Sicherheit dreifach laufen (Skalierung), andere brauchen viel RAM.

Das Problem: Ein Administrator kann nicht mehr manuell per SSH auf jeden Server gehen, docker run tippen, freie Ports suchen und die Auslastung überwachen.

Die Lösung: Ein Orchestrator übernimmt die vollständige Kontrolle über einen Verbund von Servern (Cluster) und verwaltet die Container autonom.

Container mit Docker, Scheduling & Orchestrierung

Die Kernkomponente – Der Scheduler (Die Zuweisung)

Was macht ein "Scheduler"?

Aufgabe: Der Scheduler ist der Logistik-Manager des Systems. Er entscheidet, auf welchem konkreten physischen Server ein neuer Container gestartet wird.

Wie trifft er die Entscheidung? (Kriterien):

Ressourcen-Anforderungen: Benötigt der Container 4 GB RAM, setzt der Scheduler ihn nur auf einen Server, der diesen Platz noch frei hat.

Data Locality: Muss der Container nah an einer bestimmten Datenbank liegen?

Anti-Affinität: Der Scheduler sorgt aktiv dafür, dass die drei Kopien deines "Noten-Service" auf drei verschiedenen Servern landen. Schmilzt ein Server ab, laufen die anderen zwei weiter.

Container mit Docker, Scheduling & Orchestrierung

Die **Aufgaben des Orchestrators** (Das Management)

Die Rundum-Sorglos-Infrastruktur

Ein moderner Orchestrator bietet drei fundamentale Kernfeatures:

Self-Healing (Selbstheilung): Stürzt ein Container oder ein kompletter Server ab, bemerkt das der Orchestrator sofort. Er startet die betroffenen Container automatisch auf einem gesunden Server neu.

Declarative State (Soll-Zustand): Der Admin schreibt eine Datei: "Ich möchte, dass immer genau 5 Instanzen des Frontends laufen." Der Orchestrator gleicht den Ist-Zustand permanent an diesen Soll-Zustand an.

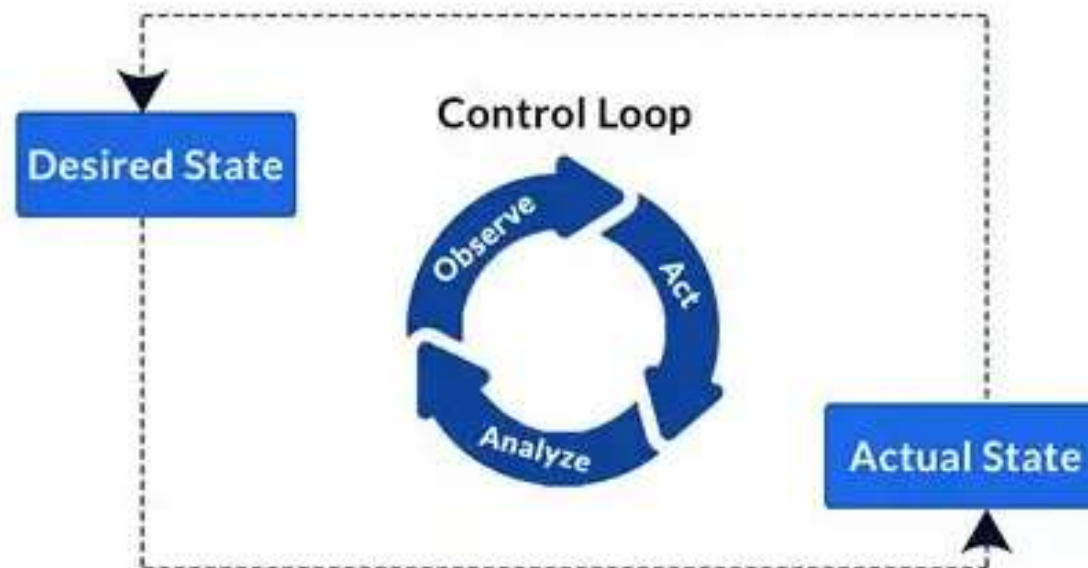
Automated Rollouts & Rollbacks: Updates werden schrittweise eingespielt (z. B. immer nur ein Container nach dem anderen). Gibt es Fehler, bricht das System ab und wechselt automatisch zur alten Version.

Container mit Docker, Scheduling & Orchestrierung

Die **Aufgaben des Orchestrators** (Das Management)

Soll / Ist-Vergleich in der Kontrollschleife

Quelle: <https://k21academy.com/kubernetes/kubernetes-operator/>



Container mit Docker, Scheduling & Orchestrierung

Die Architektur eines Cluster-Managers

Wie ist ein Container-Cluster aufgebaut?

Die Struktur teilt sich immer in zwei Rollen auf:

Die **Master-Nodes** (Die Zentrale): Hier läuft das "Gehirn" des Clusters (inklusive Scheduler und dem Key-Value-Store wie etcd). Sie treffen alle Entscheidungen, lassen selbst aber keine Kunden -Anwendungen laufen.

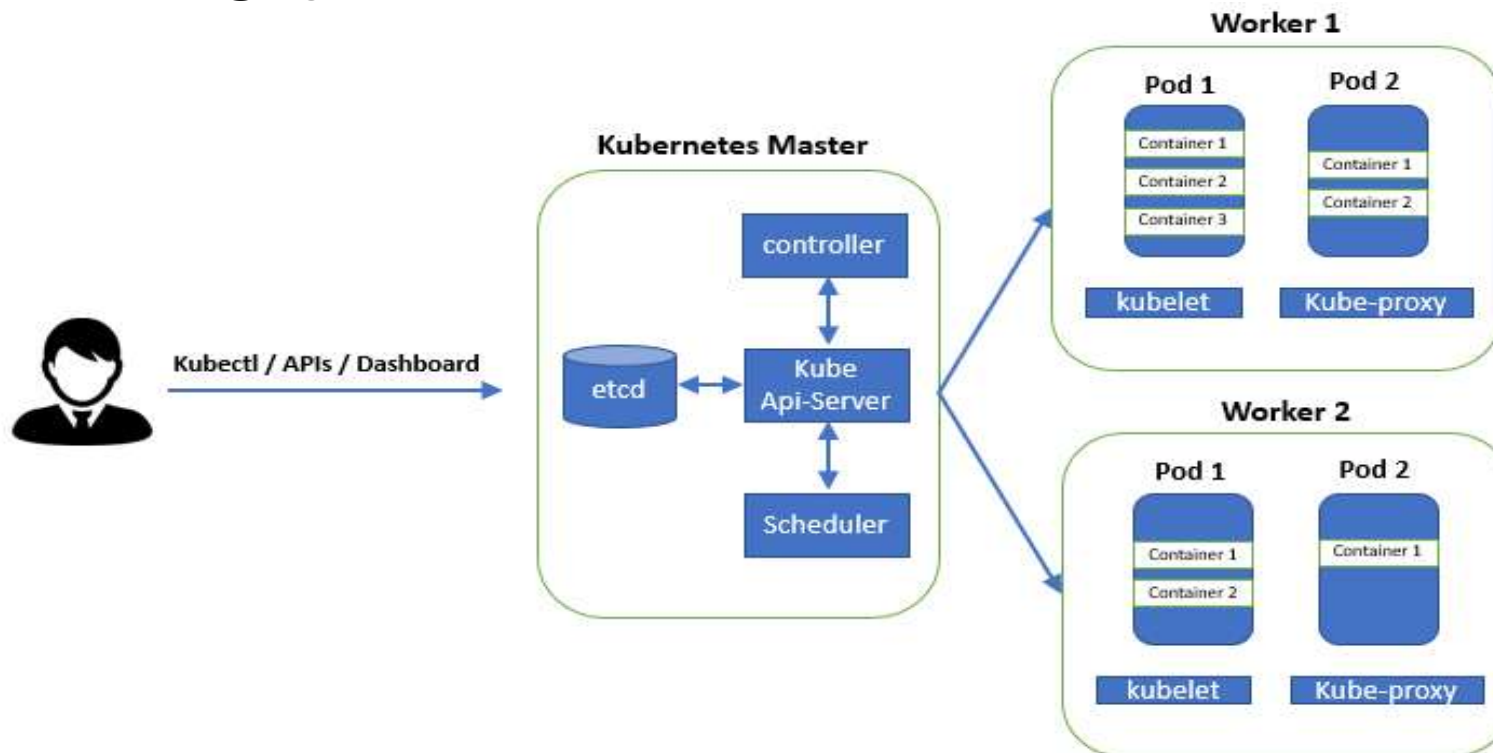
Die **Worker-Nodes** (Die Arbeiter): Das sind die Server, die die eigentliche Arbeit verrichten. Auf ihnen läuft die Docker-Engine (oder containerd) und sie führen die Container (Pods) aus.

Container mit Docker, Scheduling & Orchestrierung

Die Architektur eines Cluster-Managers

Wie ist ein Container-Cluster aufgebaut?

Quelle: <https://medium.com/@srghimire061/how-to-install-kubernetes-on-master-and-worker-nodes-ec2f6dabdbdd>



Container mit Docker, Scheduling & Orchestrierung

Die bekanntesten Orchestratoren im Vergleich

Die drei wichtigsten Tools im Überblick:

Kubernetes (K8s): Der unangefochtene Industriestandard, ursprünglich von Google entwickelt. Extrem mächtig, hochgradig flexibel, aber für Einsteiger komplex zu erlernen.

Docker Swarm: Die native Lösung, die direkt in Docker integriert ist. Nutzt dieselbe Syntax wie docker-compose. Perfekt für kleinere bis mittlere Setups

Nomad: Eine schlanke und flexible Alternative von HashiCorp, die nicht nur Container, sondern auch klassische Java- oder Binärdateien orchestrieren kann.